

LF09:15:Sockets & Ports (& Services)

für die Ausbildung zur Fachinformatikerin

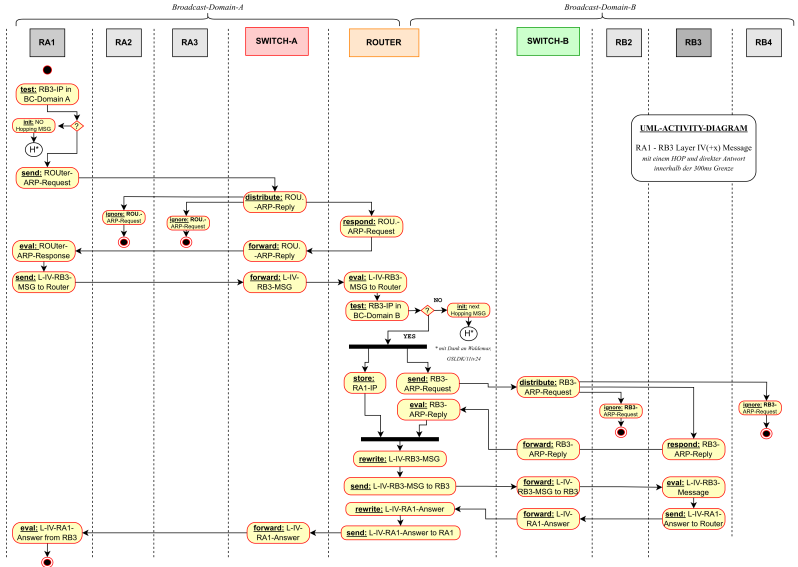
Karsten Reincke

Gewerbliche Schulen Dillenburg

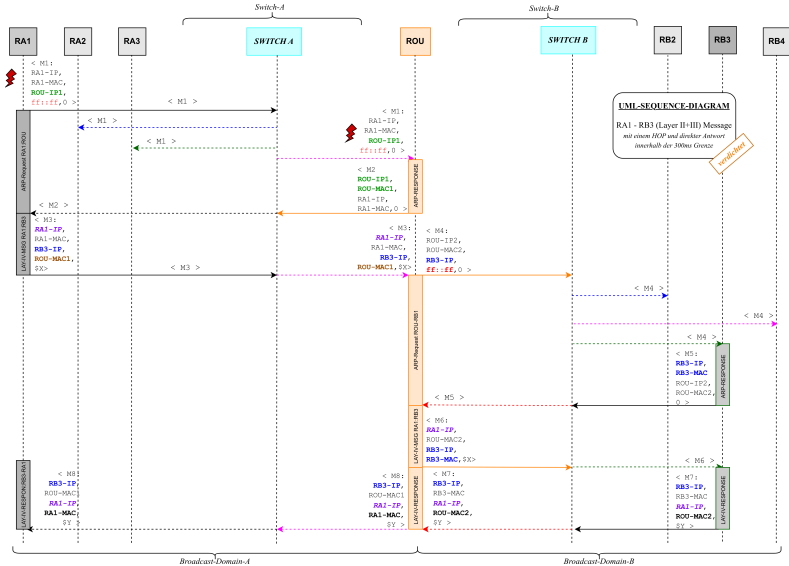
11. August 2026

*Diese Präsentation stammt aus dem OER-Projekt **proTirone**. Es wurde auf Basis von **proScientia.ltz** entwickelt und ist **CC-BY-4.0** lizenziert. Gemäß CC-BY-4.0/§5 werden proTirone-Materialien so wie sie sind angeboten, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung.*

LF09:15:L3-to-L3-with-Hopping:Activity Reminder

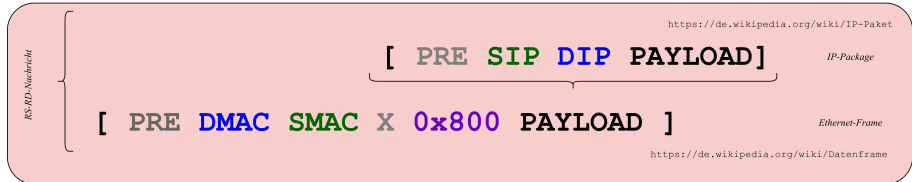
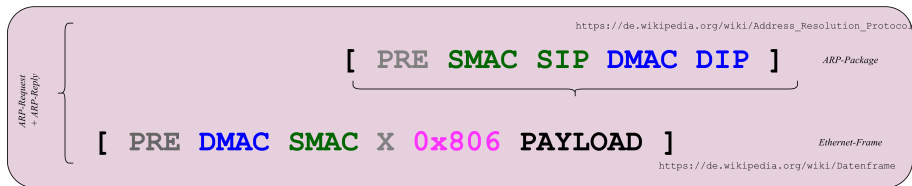
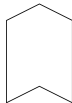


LF09:15:L3-to-L3-with-Hopping:Sequence Reminder



LF09:15:L3-to-L3-with-Hopping:Package Reminder

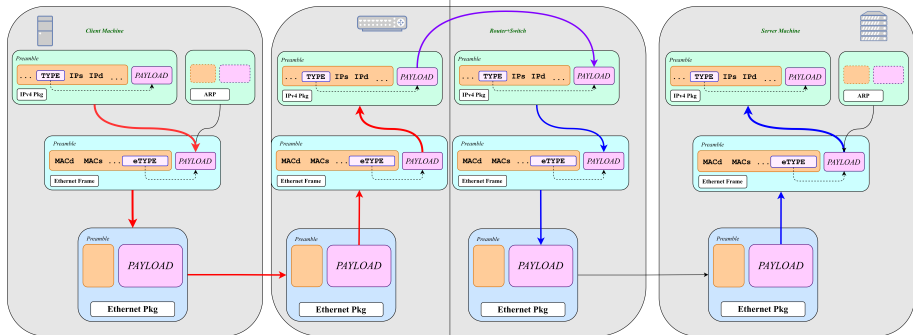
[RSIP RSMAC RDIP RDMAC PL]



LF09:15:L3-to-L3-with-Hopping:Embedded Packages

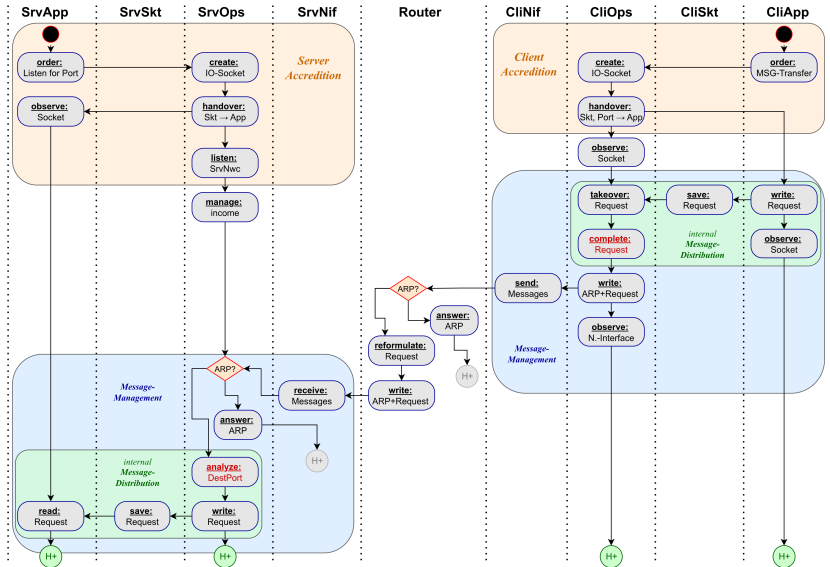
Broadcast-Domain A

Broadcast-Domain B

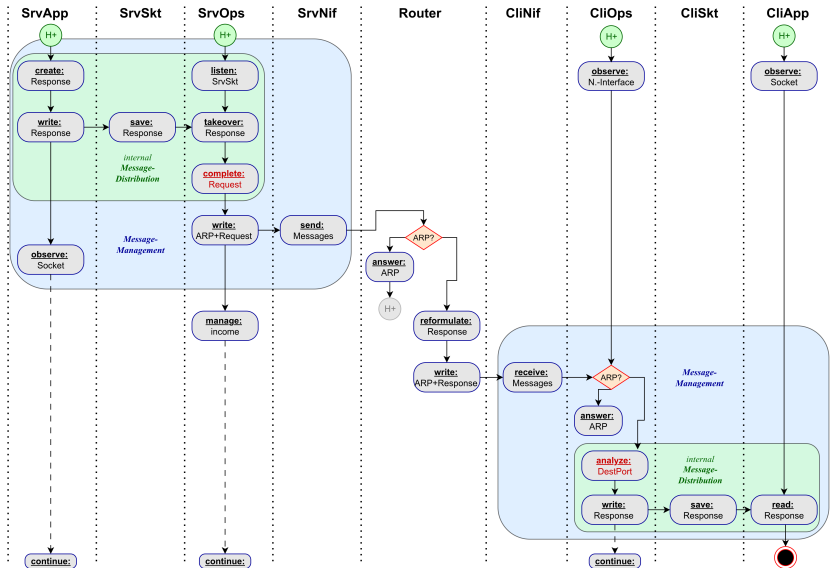




LF09:15:L7-to-L7-with-Hopping:Request

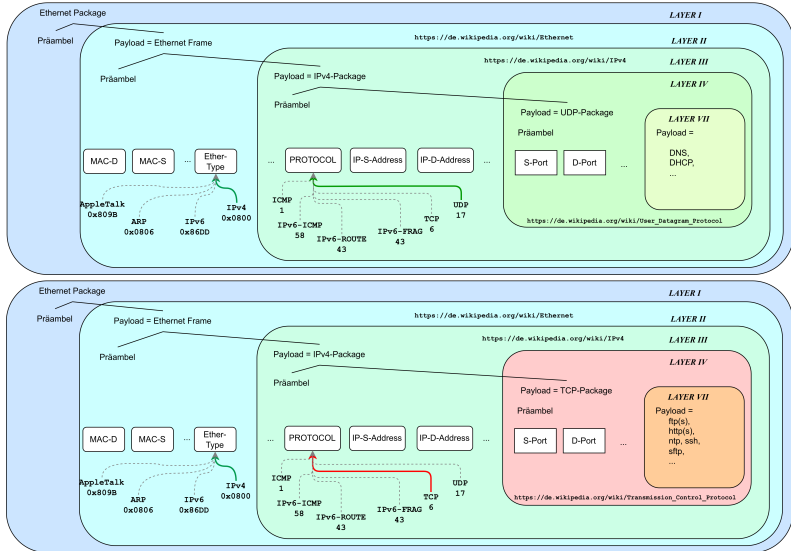


LF09:15:L7-to-L7-with-Hopping:Response



LF09:15:L7-to-L7-with-Hopping:Embedded Packages

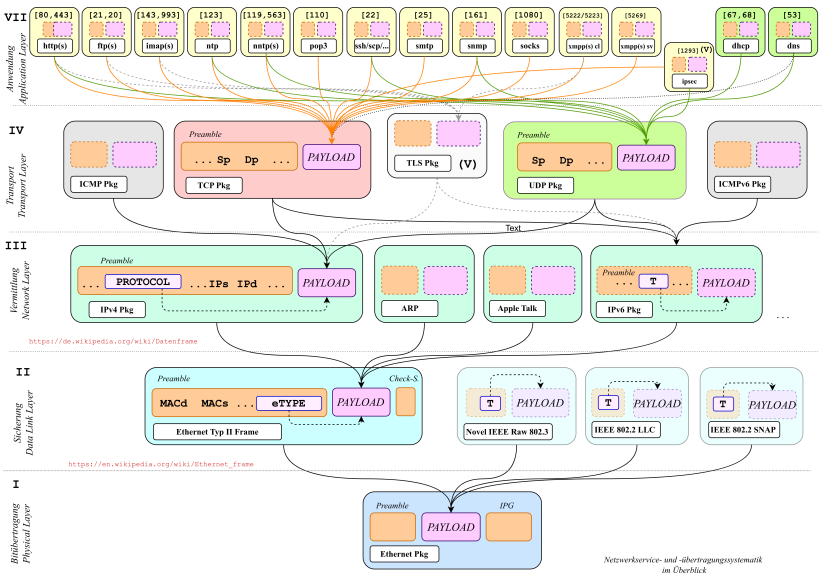
Kooperation von Package-Type, Adressen, Service-Typ, und Port





- Systemports := 0 - 1023 ('well-known ports', registriert bei der IANA).
 - 7 ECHO-Service
 - 20 FTP (Datenübertragung: nur TCP)
 - 21 FTP (Verbindungsaufbau und Steuerung: TCP, UDP)
 - 22 SSH (TCP, UDP)
 - 23 Telnet (unverschlüsseltes Textprotokoll, z. B. für Fernwartung)
 - 25 SMTP (Simple Mail Transfer Protocol)
 - 67 Bootstrap Protocol (auch von DHCP genutzt: nur UDP)
 - 80 HTTP (TCP, UDP)
 - 110 POP3 (Post Office Protocol 3 = Mailclient zu Mailserver)
 - 123 NTP (Network Time Protocol)
 - 143 IMAP (Internet Message Access Protocol = Mailclient zu Mailserver)
 - 319 PTP (Precision Time Protocol):Events 320 PTP:General Messages (nur UDP)
 - 443 HTTPS (Hypertext Transfer Protocol over SSL/TLS: TCP, UDP)
- Registrierte Ports := 1024 - 49151 (per Konvention / 'Gebrauchsrecht')
 - 1434 MSSQL
 - 1883 MQTT
 - 3306 Mysql
 - 8080 HTTP über Poxysver als Vermittler zwischen Client und realen Server

LF09:15:Packages,Ports&Services::Package System



<https://de.wikipedia.org/wiki/Datenframe>

https://en.wikipedia.org/wiki/Ethernet_frame

Netzwerkservice- und -übertragungssystematik
im Überblick



LF09:15:Packages,Ports&Services:Echo:Client

```
1  # (C) 2025 K.Reincke: proTirone snippet [CC-BY-4.0]
2  # in broad terms inspired by https://realpython.com/python-sockets/
3
4  import socket, time
5
6  server_address = "127.0.0.1" # echo server's hostname or IP address
7  server_port = 65432          # port used by the echo server
8  zzz = 10                     # sleeping time for reviewing tcp status
9
10 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
11     print(f"calling for a dynamic socket with port"); time.sleep(zzz)
12     s.connect((server_address, server_port))
13     print(f"got a socket"); time.sleep(zzz)
14
15     print(f"sending 'Hello Server' to {server_address}:{server_port}")
16     s.sendall(b"Hello Server"); time.sleep(zzz)
17     data = s.recv(1024)
18
19     print(f"Received back: {data!r}")
```



LF09:15:Packages,Ports&Services:Echo:Server

```
1  # (C) 2025 K.Reincke: proTirone snippet [CC-BY-4.0]
2  # in broad terms inspired by https://realpython.com/python-sockets/
3  import socket
4
5  server_addr = "127.0.0.1" # loopback interface address (localhost)
6  # wkp: 0 - 1023 | rp: 1024 - 49151 | dp: 49152 - 65535 | wkp echo: 7
7  server_port = 65432 # better using a 'handcrafted' dynamic port ;- )
8
9  with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
10     s.bind((server_addr, server_port))
11     s.listen()
12     print(f"acting on/for {server_addr}:{server_port}")
13     while True:
14         conn, addr = s.accept() # waiting for next request
15         print(f"requested by {addr}")
16         while True:
17             data = conn.recv(1024)
18             if not data: break
19             conn.sendall(data)
20         print(f"served response")
```



```
sequenceDiagram
    participant Client
    participant Server
    Note over Client, Server: Client sends SYN
    Client->>Server: SYN seq=x
    Note over Server: Server receives SYN
    Server->>Client: SYN-ACK ack=x+1 seq=y
    Note over Client: Client receives SYN-ACK
    Client->>Server: ACK ack=y+1 seq=x+1 [data]
    Note over Server: Server receives ACK
```

The diagram illustrates the three-way handshake process between a Client and a Server. The Client initiates the process by sending a **SYN** packet with sequence number x . The Server responds with a **SYN-ACK** packet, acknowledging the Client's sequence number as $x+1$ and sending its own sequence number y . Finally, the Client sends an **ACK** packet, acknowledging the Server's sequence number as $y+1$ and sending its own sequence number $x+1$. The **ACK** packet is shown with a gray background and contains the label **[data]**.



Herzlichen Dank für Ihre Aufmerksamkeit!



Open Educational Resources

*Dieses Dokument stammt aus dem OER-Projekt <https://github.com/protirone/>, initiiert v. **Karsten Reincke, Hohenahr**. Es stellt **freie Unterrichtsmaterialien** samt **Quellcode** unter den Bedingungen der **CC-BY-4.0-Lizenz** zur Verfügung: Sie werden so wie sie sind angeboten, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).*

Bildnachweise

-  von Wikimedia Commons. **Lizenziert** unter **CC0**. Bereitgestellt auf **Wikimedia**.
-  von Karsten Reincke. **Lizenziert** unter **proTirone-Logo-License**. Bereitgestellt auf **github**. (may only be used as logo for proTirone)
-  von GDJ. **Lizenziert** unter **CC0**. Bereitgestellt auf **Open Clipart**.